

Gym Management System

Dokumentáció



Készítette:

Csordás Bence, Fekete Ákos András, Göndör Mátyás

Tartalom

Csordás Bence, Fekete Ákos András, Göndör Mátyás	0
1.1 Témaválasztás megindoklása.....	3
1.2 Célkitűzés	3
2.1 Fejlesztői Környezet.....	4
2.1.1 Fejlesztői Eszközök - VS Code	4
2.1.2 Fejlesztői Eszközök - XAMPP	4
2.1.3 Fejlesztői eszközök - GitHub	4
2.1.4 Fejlesztői Nyelvek - PHP	5
2.1.5 Fejlesztői Nyelvek - MySQL	5
2.2 Adatbázisok szerkezetének szemléltetése	6
2.2.1 Owner tábla	6
2.2.2 Trainer tábla	6
2.2.3 Login tábla.....	7
2.2.4 Gym tábla	7
2.2.5 Memberships tábla	8
2.2.6 Trainer_gym tábla	8
2.3 Algoritmusok	8
2.3.1 loginUser algoritmus.....	9
2.3.2 Felhasználó regisztrálása algoritmus	9
2.3.3 Tulajdonos regisztrálása algoritmus	11
2.3.4 Felhasználói tagság ellenőrzése algoritmus.....	12
2.3.5 Tagság létrehozása algoritmus	13
2.4 Tesztek Dokumentálása	13
2.4.1 testPasswordsDoNotMatch	13
2.4.2 testUserAlreadyExists	13
2.4.3 testSuccessfulRegistration	15
2.4.4 testLoginSuccessful	16
2.4.5 testLoginFailed	17
2.4.6 testHasActiveMembershipTrue	18
2.4.7 testPurchaseMembershipSuccess	18
2.4.8 testPurchaseMembershipFailure.....	19
3.1 A webalkalmazás ismertetése röviden.....	21
3.2 Szükséges hardver eszközök	21

3.3 Szükséges szoftver eszközök	21
3.4 Az alkalmazás beüzemelése, működtetése	21
3.5 Program használata	25
3.6 Lehetséges hibajelzések.....	31
3.6.1 Invalid credentials.....	31
3.6.2 Passwords do not match	32
3.6.3 Username or mobile number already exists	32
3.6.4 You already have a membership for this gym	33
3.7 Elérhetőség információ kéréshez.....	33
4. Összefoglalás	34
4.1 Munka felosztása	34
4.2 A webalkalmazás jövője	34

1. Bevezetés

1.1 Témaválasztás megindoklása

Manapság az emberek nagy része ragaszkodik az egészséges életmódhoz és ezzel a webalkalmazással, egyszerűen választhatnak megfelelő edzőtermet.

1.2 Célkitűzés

A GMS egy felhasználóbarát webalkalmazás amely különböző edzőtermek gyűjtő oldala, segítségével egyszerűen tudnak az emberek tájékozódni a Kaposváron és környékén lévő konditermek közül. A weboldal segítségével online is tudnak az emberek bérletet venni, személyi edzőt választani időpontot foglalni. A webalkalmazás segítségével edzőtermek tulajdonosai hozhatnak létre és kezelhetik edzőtermeiket, melyekhez hozzá rendelhetnek személyi edzőket.

2. Fejlesztői Dokumentáció

2.1 Fejlesztői Környezet

A kód többnyire Visual Studio Code-ban készült. Az adatbáziskezeléshez MySQL-t és PHP-t használtunk az XAMPP segítségével, ez lehetővé teszi hogy a felhasználók adatait könnyen eltudjuk tárolni és biztonságosan megtudjuk őrizni. A verziókezeléshez GitHubot és annak az asztali alkalmazását (GitHub desktop) használtuk.

2.1.1 Fejlesztői Eszközök - VS Code

A Visual Studio Code (VS Code) egy ingyenes, nyílt forráskódú kódszerkesztő, amit a Microsoft fejleszt. Nagyon népszerű a fejlesztők körében, mert könnyű, gyors, mégis nagyon sokoldalú. Itt van pár fontos dolog róla: Forráskód szerkesztésére: szinte bármilyen programozási nyelvet támogat (pl. Python, JavaScript, C++, HTML, stb.). Hibakeresés (debugging): beépített eszközökkel lehet lépésenként végigmenni a kódon. Verziókezelés (Git): beépített Git támogatás van benne, így egyszerűen kezelheted a kódváltoztatásokat.

2.1.2 Fejlesztői Eszközök - XAMPP

Az XAMPP egy ingyenes, egyszerűen telepíthető szoftvercsomag, ami egy komplett webszerver-környezetet biztosít a számítógépeden. Kifejezetten hasznos, ha webfejlesztéssel foglalkozol, és helyben (azaz az internet nélkül) szeretnél kipróbálni vagy fejleszteni weboldalakat.

Használata:

Helyi weboldal fejlesztéshez – nem kell tárhelyet bérelned, csak elindítod az XAMPP-ot és már működik is a weboldalad a saját gépeden.

PHP kód futtatása – például WordPress, Laravel vagy más PHP alapú rendszerek teszteléséhez.

Adatbázisok kezelése – jön vele egy phpMyAdmin nevű eszköz, amivel könnyen kezelheted az adatbázisaidat.

2.1.3 Fejlesztői eszközök - GitHub

A GitHub egy weboldal és szolgáltatás, ahol programkódot lehet tárolni, kezelni és megosztani – főleg a Git nevű verziókezelő rendszer segítségével. Olyan, mint egy szociális hálózat fejlesztőknek, de főleg a közös munkát segíti.

Mire jó a GitHub?

1. Kód feltöltése és mentése (verziókezeléssel)
2. Több ember közös munkája ugyanazon a projekten

3. Változások követése: ki mit módosított, mikor, és miért
4. Nyílt forráskódú projektek megosztása
5. Hibák kezelése, új funkciók javaslása

2.1.4 Fejlesztői Nyelvek - PHP

A PHP egy programozási nyelv, amit főként webfejlesztésre használnak. A név eredetileg a „Personal Home Page” rövidítése volt, de ma már inkább a „PHP: Hypertext Preprocessor” rekurzív rövidítést használják.

Célja:

1. Dinamikus weboldalak készítése (pl. belépési rendszerek, űrlapkezelés, adatbázis-kapcsolat).
2. Szerveroldali programozás – tehát a kód nem a böngészőben, hanem a webszerveren fut le.
3. Könnyen összekapcsolható adatbázisokkal, például MySQL-lel.

2.1.5 Fejlesztői Nyelvek - MySQL

A MySQL egy adatbázis-kezelő rendszer, amit arra használnak, hogy adatokat tároljanak, kezeljenek és lekérdezzenek. Leggyakrabban webalkalmazások mögött fut – például egy webshop vagy blog minden bejegyzését, felhasználói fiókjait, rendeléseit MySQL-ben tárolhatják.

Mire jó a MySQL?

1. Adatok tárolására (pl. felhasználók, hozzászólások, termékek)
2. Keresésre az adatok között
3. Adatok beszúrására, módosítására, törlésére

2.1.6 Fejlesztői Nyelvek – HTML

A HTML a weboldalak alapvető építőköve – a struktúrát adja meg egy weboldalnak. A teljes neve: HyperText Markup Language.

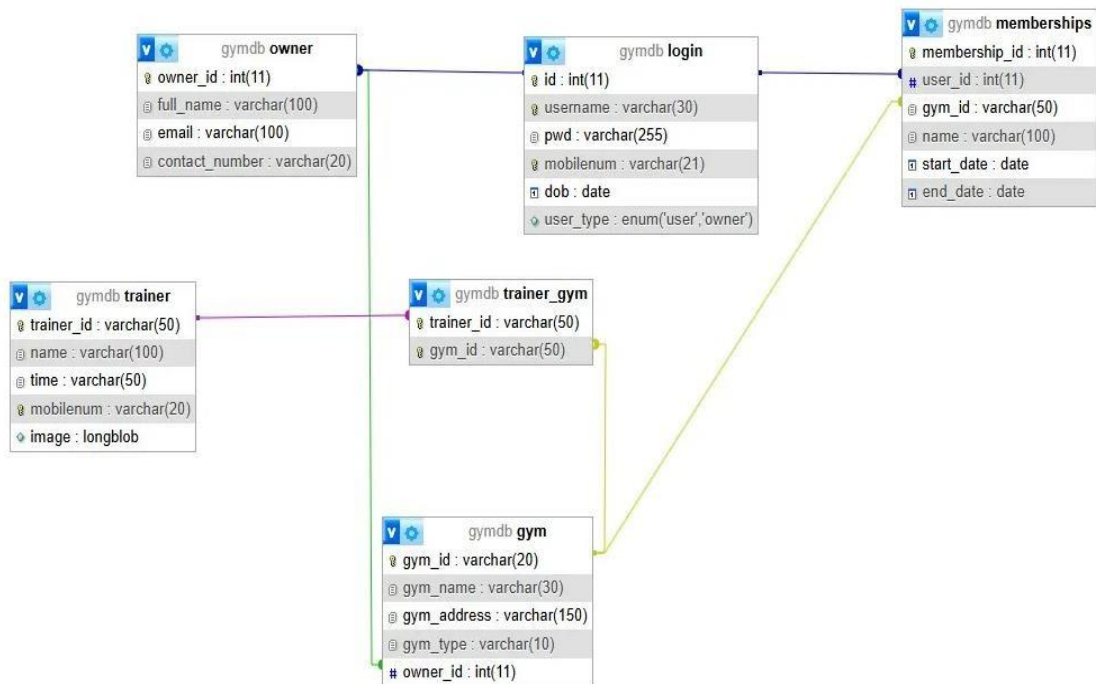
Mire jó a HTML?

- Meghatározza, mi jelenik meg egy weboldalon: szöveg, képek, linkek, űrlapok, stb.
- Nem programozási nyelv, hanem leíró nyelv – nem számol, nem dönt, csak „mondja”, hogy mi hol legyen és milyen szerepe van.

Hogyan kapcsolódik a PHP-hoz és MySQL-hez?

1. HTML: amit a felhasználó lát
2. PHP: amit a szerver csinál (pl. ellenőrzi a bejelentkezést)
3. MySQL: ahol az adatok vannak (pl. jelszavak, nevek)

2.2 Adatbázisok szerkezetének szemléltetése



A táblák közötti kapcsolatok

2.2.1 Owner tábla

owner_id: Ez a mező tulajdonosok azonosítóját tárolja, ez az azonosító az elsődleges kulcs és INT típusú, maximum 11 karaktert fogad be.

full_name: Tulajdonosok teljes nevét tárolja. Varchar típusú és maximum 100 karaktert fogad be.

email: Tulajdonosok e-mail címét tárolja. Varchar típusú és maximum 100 karaktert fogad be.

contact_number: Tulajdonosok telefonszámát tárolja. Varchar típusú és maximum 20 karaktert fogad be.

gymdb owner
owner_id : int(11)
full_name : varchar(100)
email : varchar(100)
contact_number : varchar(20)

2.2.2 Trainer tábla

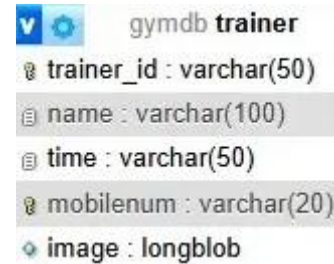
trainer_id: Az edző azonosítóját tárolja. Varchar típusú és maximum 50 karaktert fogad be.

name: Az edző nevét tárolja. Varchar típusú és maximum 100 karaktert fogad be.

time: Az edző elérhetőségét tárolja. Varchar típusú és maximum 50 karaktert fogad be.

mobilenum: Az edző telefonszámát tárolja. Varchar típusú és maximum 20 karaktert fogad be.

image: Az edzőről egy képet tárol el. Longblob típusú.



🔑	trainer_id	: varchar(50)
📄	name	: varchar(100)
📄	time	: varchar(50)
🔑	mobilenum	: varchar(20)
📄	image	: longblob

2.2.3 Login tábla

id: A felhasználó azonosítóját tárolja. Int típusú és maximum 11 karaktert fogad be.

username: A felhasználó felhasználónevét tárolja. Varchar típusú és maximum 30 karaktert fogad be.

pwd: A felhasználó jelszavát tárolja. Varchar típusú és maximum 255 karaktert fogad be.

mobilenum: A felhasználó telefonszámát tárolja. Varchar típusú és maximum 21 karaktert fogad be. dob: A felhasználó születési dátumát tárolja. Date típusú.

user_type: A felhasználó típusát tárolja (user vagy owner). enum típusú.



🔑	id	: int(11)
📄	username	: varchar(30)
📄	pwd	: varchar(255)
🔑	mobilenum	: varchar(21)
📄	dob	: date
📄	user_type	: enum('user', 'owner')

2.2.4 Gym tábla

Gym_id: Az edzőterem azonosítóját tárolja. Varchar típusú és maximum 20 karaktert fogad be.

Gym_name: Az edzőterem nevét tárolja. Varchar típusú és maximum 30 karaktert fogad be.

Gym_address: Az edzőterem címét tárolja. Varchar típusú és maximum 150 karaktert fogad be.

Gym_type: Az edzőterem típusát tárolja. Varchar típusú és maximum 10 karaktert fogad be.

Owner_id: Az edzőterem tulajdonosának az azonosítóját tárolja. Int típusú és maximum 11 karaktert fogad be.



🔑	gym_id	: varchar(20)
📄	gym_name	: varchar(30)
📄	gym_address	: varchar(150)
📄	gym_type	: varchar(10)
#	owner_id	: int(11)

2.2.5 Memberships tábla

Membership_id: A bérlet azonosítóját tárolja. Int típusú és maximum 11 karaktert fogad be.

User_id: A felhasználó azonosítóját tárolja és hozzá rendeli a megadott bérletet.

Gym_id: A megadott edzőteremhez hozzáad egy bérletet a megfelelő felhasználónak.

Name: Eltárolja, hogy milyen néven legyen a bérlet (felhasználó teljes neve).

Start_date: A megvásárolt bérlet kezdeti idejét tárolja. Date típusú.

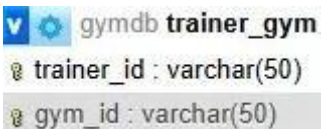
End_date: A megvásárolt bérlet lejárat dátumát tárolja. Date típusú.



gymdb memberships	
🔑	membership_id : int(11)
#	user_id : int(11)
🔑	gym_id : varchar(50)
	name : varchar(100)
📅	start_date : date
📅	end_date : date

2.2.6 Trainer_gym tábla

Ez a tábla hozzárendeli a megadott személyi edzőket a megfelelő edzőtermekhez.



gymdb trainer_gym	
🔑	trainer_id : varchar(50)
🔑	gym_id : varchar(50)

2.3 Algoritmusok

Az alábbi kategóriában 4 különböző algoritmus működése látható

2.3.1 loginUser algoritmus

```
function loginUser($conn, $username, $password): mixed {  
    $username = trim(string: $username);  
    $password = trim(string: $password);  
  
    $stmt = $conn->prepare("SELECT id, username, user_type FROM login WHERE username = ? AND pwd = ?");  
    if (!$stmt) {  
        return "Database error.";  
    }  
  
    $stmt->bind_param("ss", $username, $password);  
  
    if (!$stmt->execute()) {  
        return "Database error during execute.";  
    }  
  
    $result = $stmt->get_result();  
    if (!$result) {  
        return "Database error getting result.";  
    }  
  
    if ($user = $result->fetch_assoc()) {  
        return $user;  
    } else {  
        return "Invalid credentials";  
    }  
}
```

Ez az algoritmus a bejelentkezést biztosítja. Levágja a szóközöket a username és password körül trim()-el, előkészíti az SQL lekérdezést prepare()-el, ha ez nem sikerül "Database error" hibaüzenettel tér vissza, ezután hozzáköti az adatokat a lekérdezéshez bind_param()-al, majd futtatja a lekérdezést execute()-al, ami hiba esetén "Database error during execute" üzenettel tér vissza, megszerzi a lekérdezés eredményét get_result()-al aminél, ha hibát észlel, "Database error getting result" üzenettel tér vissza. Végül ellenőrzi az adatokat, találat esetén visszaadja a felhasználó adatait (id, username, user_type) tömb formában, ha nincs találat, "Invalid credentials" üzenettel tér vissza.

2.3.2 Felhasználó regisztrálása algoritmus

```

function registerUser($conn, $username, $password, $confirm_password, $mobilenum, $dob): bool|string {
    $username = trim(string: $username);
    $password = trim(string: $password);
    $confirm_password = trim(string: $confirm_password);
    $mobilenum = trim(string: $mobilenum);
    $dob = trim(string: $dob);

    if ($password !== $confirm_password) {
        return "Passwords do not match!";
    }

    $check_user = "SELECT * FROM login WHERE username = ? OR mobilenum = ?";
    $stmt = $conn->prepare($check_user);
    if (!$stmt) {
        return "Database error during SELECT prepare.";
    }
    $stmt->bind_param("ss", $username, $mobilenum);

    if (!$stmt->execute()) {
        return "Database error during SELECT execute.";
    }

    $result = $stmt->get_result();
    if ($result && $result->num_rows > 0) {
        return "Username or mobile number already exists. Choose another!";
    }

    $insert_user = "INSERT INTO login (username, pwd, mobilenum, dob) VALUES (?, ?, ?, ?)";
    $stmt = $conn->prepare($insert_user);
    if (!$stmt) {
        return "Database error during INSERT prepare.";
    }
    $stmt->bind_param("ssss", $username, $password, $mobilenum, $dob);

    if ($stmt->execute()) {
        return true;
    } else {
        return "Registration failed. Try again!";
    }
}

```

Ez az algoritmus a felhasználó regisztrációt biztosítja. Levágja a szóközöket a username, password, confirm_password, mobilenum és dob körül trim()-el. Ezután ellenőrzi, hogy a jelszó és a megerősítő jelszó egyezik-e, ha nem, akkor "Passwords do not match!" hibaüzenettel tér vissza. Előkészíti az SQL lekérdezést prepare()-el, amely ellenőrzi, hogy a felhasználónév vagy a mobilszám már szerepel-e az adatbázisban. Ha a prepare() nem sikerül, "Database error during SELECT prepare." üzenettel tér vissza. Az adatokat hozzáköti a lekérdezéshez bind_param()-mal, majd lefuttatja execute()-al. Ha az execute() nem sikerül, "Database error during SELECT execute." hibaüzenettel tér vissza. Ezután az eredményt megszerzi get_result()-al, és ha talál már meglévő felhasználót, akkor "Username or mobile number already exists. Choose another!" hibaüzenetet ad vissza. Ha nincs ütközés, új SQL lekérdezést készít az új felhasználó beszúrására a login táblába. Itt ismét prepare()el előkészíti az INSERT-et, hibás prepare() esetén "Database error during INSERT prepare." üzenettel tér vissza. Ezután az adatokat hozzáköti az új lekérdezéshez bind_param()-

mal, és lefuttatja execute()-al. Ha a beszúrás sikeres, true értékkel tér vissza, ha nem sikerül, "Registration failed. Try again!" hibaüzenettel fejezi be.

2.3.3 Tulajdonos regisztrálása algoritmus

```
function registerOwner($conn, $username, $password, $confirm_password, $fullname, $email, $mobilenum, $dob, $mockInsertId = null): bool|string {
    if ($password !== $confirm_password) {
        return "Passwords do not match!";
    }

    // Check if username or mobile already exists
    $check_user = "SELECT * FROM login WHERE username = ? OR mobilenum = ?";
    $stmt = $conn->prepare($check_user);
    if (!$stmt) {
        return "Error preparing SELECT user.";
    }
    $stmt->bind_param("ss", $username, $mobilenum);

    if (!$stmt->execute()) {
        return "Database error during SELECT execute.";
    }

    $result = $stmt->get_result();
    if (!$result) {
        return "Database error getting SELECT result.";
    }

    if ($result->num_rows > 0) {
        return "Username or mobile number already exists.";
    }

    // Insert into login table
    $insert_login = "INSERT INTO login (username, pwd, mobilenum, dob, user_type) VALUES (?, ?, ?, ?, 'owner')";
    $stmt = $conn->prepare($insert_login);
    if (!$stmt) {
        return "Error preparing INSERT login.";
    }
    $stmt->bind_param("ssss", $username, $password, $mobilenum, $dob);

    if (!$stmt->execute()) {
        return "Registration failed at login insertion.";
    }

    // Get the new owner ID
    if ($mockInsertId !== null) {
        $owner_id = $mockInsertId;
    } else {
        $owner_id = $conn->insert_id;
    }

    // Insert into owner table
    $insert_owner = "INSERT INTO owner (owner_id, full_name, email, contact_number) VALUES (?, ?, ?, ?)";
    $stmt_owner = $conn->prepare($insert_owner);
    if (!$stmt_owner) {
        return "Error preparing INSERT owner.";
    }
    $stmt_owner->bind_param("iiss", $owner_id, $fullname, $email, $mobilenum);

    if ($stmt_owner->execute()) {
        return true;
    } else {
        return "Registration failed at owner insertion.";
    }
}
```

Ez az algoritmus a tulajdonos (owner) regisztrációt biztosítja. Levágja a szóközöket a username, password, confirm_password, fullname, email, mobilenum és dob körül trim()-el. Ezután ellenőrzi, hogy a jelszó és a megerősítő jelszó egyezik-e, ha nem, akkor "Passwords do not match!" hibaüzenettel tér vissza. Előkészíti az SQL lekérdezést prepare()-el, amely ellenőrzi, hogy az adott felhasználónév vagy mobilszám már szerepel-e az adatbázisban. Ha a prepare() nem sikerül, "Error preparing SELECT user." üzenettel tér vissza. Az adatokat hozzáköti a lekérdezéshez bind_param()-al, majd lefuttatja execute()-al. Ha a execute() nem sikerül, "Database error during SELECT execute." üzenettel tér vissza. Megpróbálja megszerezni a lekérdezés eredményét get_result()-al, ha ez nem sikerül, "Database error getting

SELECT result." üzenetet ad vissza. Ha talál egyezést (`num_rows > 0`), akkor "Username or mobile number already exists." üzenettel tér vissza. Ha nincs meglévő felhasználó, előkészít egy új beszúrás a login táblába `prepare()`-el. Ha a `prepare()` hibás, "Error preparing INSERT login." üzenettel tér vissza. A beszúrandó adatokat hozzáköti `bind_param()`-al, majd lefuttatja `execute()`-al. Ha a beszúrás nem sikerül, "Registration failed at login insertion." üzenettel tér vissza. Ezután megállapítja az új felhasználó azonosítóját (`owner_id`): ha tesztelés történik, akkor a `mockInsertId` értékét veszi, egyébként a valós adatbázis `insert_id` értéket. Ezután új beszúrás készíti az owner táblába `prepare()`-el, ahol az új owner adatait tárolja el. Ha a `prepare()` hibás, "Error preparing INSERT owner." üzenettel tér vissza. A beszúrandó adatokat `bind_param()`-al beköti, majd lefuttatja az `execute()`-t. Ha a beszúrás sikeres, `true` értékkel tér vissza, ha nem sikerül, "Registration failed at owner insertion." üzenetet ad vissza.

2.3.4 Felhasználói tagság ellenőrzése algoritmus

```
function hasActiveMembership($conn, $user_id, $gym_id): bool {  
    $stmt = $conn->prepare("  
        SELECT membership_id FROM memberships  
        WHERE user_id = ? AND gym_id = ? AND end_date >= CURDATE()  
    ");  
    if (!$stmt) {  
        return false;  
    }  
    $stmt->bind_param("is", $user_id, $gym_id);  
    $stmt->execute();  
    $result = $stmt->get_result();  
    return $result->num_rows > 0;  
}
```

Ez az algoritmus azt ellenőrzi, hogy egy felhasználónak van-e aktív tagsága egy adott edzőteremben. Először előkészíti az SQL lekérdezést `prepare()`-el, amely a memberships táblából keres, olyan feltételekkel, hogy a `user_id` és a `gym_id` egyezzen, és az `end_date` dátum még ne legyen lejárt. Ha a `prepare()` nem sikerül, akkor `false` értékkel tér vissza. A lekérdezés paramétereit hozzáköti `bind_param()`-al, majd végrehajtja a lekérdezést `execute()`-al. Ezután megszerzi az eredményt `get_result()`-al, majd ellenőrzi, hogy van-e találat (`num_rows > 0`). Ha talál aktív tagságot, akkor `true`-t ad vissza, ha nem, akkor `false`-t.

2.3.5 Tagság létrehozása algoritmus

```
function purchaseMembership($conn, $user_id, $gym_id, $name, $duration_days): mixed {
    $start_date = date(format: "Y-m-d");
    $end_date = date(format: "Y-m-d", timestamp: strtotime(datetime: "+$duration_days days"));

    $stmt = $conn->prepare("INSERT INTO memberships (user_id, gym_id, name, start_date, end_date) VALUES (?, ?, ?, ?, ?)");
    if (!$stmt) {
        return false;
    }
    $stmt->bind_param("issss", $user_id, $gym_id, $name, $start_date, $end_date);
    return $stmt->execute();
}
```

Ez az algoritmus egy új tagságot hoz létre egy felhasználó számára. Először meghatározza a tagság kezdő dátumát a mai napnak, majd kiszámolja a tagság lejáratí dátumát a megadott napok számával. Ezután előkészíti az SQL beszűrő lekérdezést prepare()-el, amellyel a memberships táblába szúrja be a felhasználó ID-t, edzőterem ID-t, nevet, kezdő dátumot és lejáratí dátumot. Ha a prepare() nem sikerül, false értékkel tér vissza. A beszűrő adatokat hozzáköti a lekérdezéshez bind_param()-mal, majd lefuttatja a beszűrőst execute()-al. Végül az execute() eredményét adja vissza: ha sikeres, true, ha nem sikerül, false.

2.4 Tesztek Dokumentálása

2.4.1 testPasswordsDoNotMatch

```
public function testPasswordsDoNotMatch(): void
{
    $connMock = $this->createMock(mysqli::class);
    $result = registerUser(conn: $connMock, username: "user", password: "pass1", confirm_password: "pass2", "123", "2000-01-01");
    $this->assertEquals("Passwords do not match!", $result);
}
```

Teszteset: testPasswordsDoNotMatch() Cél: Ez a teszteset azt vizsgálja, hogy a registerUser() függvény megfelelően kezeli-e azt az esetet, amikor a megadott jelszó (password) és a megerősítő jelszó (confirm_password) nem egyezik meg.

Lépések:

1. Létrehoz egy mockolt adatbázis kapcsolatot (\$connMock).
2. Meghívja a registerUser() függvényt eltérő jelszó és megerősítő jelszó paraméterekkel ("pass1" és "pass2").

Elvart eredmény: A függvény nem folytatja a regisztrációt, hanem a "Passwords do not match!" szöveget adja vissza.

2.4.2 testUserAlreadyExists

```

public function testUserAlreadyExists(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('get_result')->willReturn(new class {
        public $num_rows = 1;
    });

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $stmtMock->method('bind_param')->willReturn(true);
    $stmtMock->method('execute')->willReturn(true);

    $result = registerUser($connMock, username: "existinguser", password: "pass", confirm_password: "pass", "123", "2000-01-01");

    $this->assertEquals("Username or mobile number already exists. Choose another!", $result);
}

```

Teszteteset: testUserAlreadyExists() Cél: Ez a teszteteset azt vizsgálja, hogy a registerUser() függvény helyesen kezeli-e azt az esetet, amikor a regisztrálni kívánt felhasználónév (username) vagy mobilszám (mobilenumber) már létezik az adatbázisban.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (\$stmtMock), amely num_rows = 1 értékkel tér vissza, ezzel szimulálva, hogy van már ilyen felhasználó.
2. Mockolja az adatbázis kapcsolatot (\$connMock), amely a prepare(), bind_param() és execute() hívásokra pozitív választ ad. Meghívja a registerUser() függvényt egy már létező felhasználó adataival (existinguser).

Elvárt eredmény: A függvény nem engedi újragisztrálni a meglévő felhasználót, hanem figyelmeztető hibaüzenetet ad vissza.

2.4.3 testSuccessfulRegistration

```
public function testSuccessfulRegistration(): void
{
    $stmtCheckMock = $this->createMock(mysqli_stmt::class);
    $stmtCheckMock->method('get_result')->willReturn(new class {
        public $num_rows = 0;
    });
    $stmtCheckMock->method('execute')->willReturn(true);

    $stmtInsertMock = $this->createMock(mysqli_stmt::class);
    $stmtInsertMock->method('execute')->willReturn(true);

    $connMock = $this->createMock(mysqli::class);

    $connMock->method('prepare')->willReturnCallback(function($query) use ($stmtCheckMock, $stmtInsertMock): mixed {
        if (str_starts_with($query, 'SELECT')) {
            return $stmtCheckMock;
        } else {
            return $stmtInsertMock;
        }
    });

    $result = registerUser($connMock, username: "newuser", password: "pass", confirm_password: "pass", "123", "2000-01-01");

    $this->assertTrue($result);
}
```

Teszt eset: testSuccessfulRegistration() Cél: Ez a teszt eset azt vizsgálja, hogy a registerUser() függvény helyesen működik-e teljes sikeres regisztráció esetén, amikor sem felhasználónév, sem mobiltelefon szám ütközés nincs, és minden adatbázisművelet helyesen lefut.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (\$stmtCheckMock), amely üres eredményt (num_rows = 0) ad vissza, ezzel szimulálva, hogy még nem létezik ilyen felhasználó.
2. Létrehoz egy másik mockolt adatbázis műveletet (\$stmtInsertMock), amely a sikeres beszúrást szimulálja (execute() -> true). A mockolt adatbázis kapcsolat (\$connMock) úgy van beállítva, hogy SELECT lekérdezésre \$stmtCheckMock, INSERT lekérdezésre \$stmtInsertMock választ adjon.
3. Meghívja a registerUser() függvényt új felhasználó adataival (newuser).

Elvárt eredmény: A függvény sikeresen regisztrál egy új felhasználót, és a visszatérési érték true.

2.4.4 testLoginSuccessful

```
public function testLoginSuccessful(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('get_result')->willReturn(new class {
        public function fetch_assoc(): array {
            return [
                'id' => 1,
                'username' => 'testuser',
                'user_type' => 'member'
            ];
        }
    });
    $stmtMock->method('execute')->willReturn(true);

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $result = loginUser(conn: $connMock, username: "testuser", password: "password");

    $this->assertIsArray($result);
    $this->assertEquals('testuser', $result['username']);
}
```

Cél: Ez a teszt az azt vizsgálja, hogy a loginUser() függvény helyesen működik-e sikeres bejelentkezés esetén, amikor a felhasználónevet és jelszót helyesen adták meg.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (\$stmtMock), amely szimulálja a get_result() és fetch_assoc() működését.
2. A fetch_assoc() egy asszociatív tömböt ad vissza, amely tartalmazza a felhasználó ID-ját, nevét és típusát ('id' => 1, 'username' => 'testuser', 'user_type' => 'member'). A mockolt adatbázis műveletek (execute() és prepare()) mind sikeresek.
3. Meghívja a loginUser() függvényt helyes adatokkal (testuser és password).

Elvárt eredmény: A függvény sikeresen megtalálja a felhasználót és visszaadja az adatait tömb formájában

2.4.5 testLoginFailed

```
public function testLoginFailed(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('get_result')->willReturn(new class {
        public function fetch_assoc(): null {
            return null;
        }
    });
    $stmtMock->method('execute')->willReturn(true);

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $result = loginUser(conn: $connMock, username: "wronguser", password: "wrongpass");

    $this->assertEquals("Invalid credentials", $result);
}
```

Cél: Ez a tesztet azt vizsgálja, hogy a loginUser() függvény megfelelően kezeli-e azt az esetet, amikor a felhasználónév (username) és jelszó (password) páros nem található meg az adatbázisban, tehát a bejelentkezés sikertelen.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (\$stmtMock), amely szimulálja a get_result() és fetch_assoc() működését.
2. A fetch_assoc() itt null-t ad vissza, ezzel jelezve, hogy nem talált felhasználót. A mockolt adatbázis műveletek (execute() és prepare()) sikeresek.
3. Meghívja a loginUser() függvényt hibás bejelentkezési adatokkal (wronguser és wrongpass).

Elvárt eredmény: A függvény felismeri, hogy nincs találat az adatbázisban, és "Invalid credentials" üzenetet ad vissza

2.4.6 testHasActiveMembershipTrue

```
public function testHasActiveMembershipTrue(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('execute')->willReturn(true);
    $stmtMock->method('get_result')->willReturn(new class {
        public $num_rows = 1;
    });

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $hasMembership = hasActiveMembership(conn: $connMock, user_id: 1, gym_id: 1);

    $this->assertTrue($hasMembership);
}
```

Cél: Ez a teszteset azt vizsgálja, hogy a hasActiveMembership() függvény helyesen ismeri-e fel azt az esetet, amikor a felhasználónak van aktív tagsága egy adott edzőteremben.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (\$stmtMock), amely sikeres execute() műveletet szimulál, majd a get_result() hívásra egy olyan objektumot ad vissza, amelyben num_rows = 1, jelezve, hogy van találat (aktív tagság).
2. A mockolt adatbázis kapcsolat (\$connMock) úgy van beállítva, hogy a prepare() hívás sikeresen visszatérjen a \$stmtMock-kal.
3. Meghívja a hasActiveMembership() függvényt egy létező felhasználó és edzőterem azonosítóval (user_id = 1, gym_id = 1).

Elvart eredmény: A függvény helyesen észleli az aktív tagság meglétét és true értékkel tér vissza.

2.4.7 testPurchaseMembershipSuccess

```
public function testPurchaseMembershipSuccess(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('execute')->willReturn(true);

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $purchase = purchaseMembership(conn: $connMock, user_id: 1, gym_id: 1, name: 'Test User', duration_days: 30);

    $this->assertTrue($purchase);
}
```

Cél: Ez a teszt eset azt vizsgálja, hogy a `purchaseMembership()` függvény helyesen működik-e tagságvásárláskor, amikor minden adat helyes és az adatbázis műveletek sikeresek.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (`$stmtMock`), amely sikeres `execute()` műveletet szimulál (`willReturn(true)`).
2. Létrehoz egy mockolt adatbázis kapcsolatot (`$connMock`), amely a `prepare()` hívásra visszaadja a `$stmtMock`-ot.
3. Meghívja a `purchaseMembership()` függvényt helyes bemenetekkel (`user_id = 1, gym_id = 1, name = 'Test User', duration_days = 30`).

Elvárt eredmény: A függvény lefutása sikeres, és `true` értékkel jelzi, hogy a tagság rögzítése megtörtént az adatbázisban.

2.4.8 testPurchaseMembershipFailure

```
public function testPurchaseMembershipFailure(): void
{
    $stmtMock = $this->createMock(mysqli_stmt::class);
    $stmtMock->method('execute')->willReturn(false);

    $connMock = $this->createMock(mysqli::class);
    $connMock->method('prepare')->willReturn($stmtMock);

    $purchase = purchaseMembership(conn: $connMock, user_id: 1, gym_id: 1, name: 'Test User', duration_days: 30);

    $this->assertFalse($purchase);
}
```

Cél: Ez a teszt eset azt vizsgálja, hogy a `purchaseMembership()` függvény megfelelően kezeli-e azt az esetet, amikor a tagságvásárlás adatbázis művelete (`execute()`) sikertelen.

Lépések:

1. Létrehoz egy mockolt adatbázis lekérdezést (`$stmtMock`), amely szimulálja, hogy a `execute()` meghívása sikertelen (`willReturn(false)`).
2. Létrehoz egy mockolt adatbázis kapcsolatot (`$connMock`), amely a `prepare()` hívásra visszaadja a `$stmtMock`-ot.
3. Meghívja a `purchaseMembership()` függvényt egy tagságvásárlási próbálkozással (`user_id = 1, gym_id = 1, name = 'Test User', duration_days = 30`).

Elvárt eredmény: A függvény felismeri a beszúrási hiba tényét, és `false` értékkel tér vissza.

Tesztelői hozzáférés biztosítása:

- username: admin password: admin

- username: owner password: owner - username: user
password: user

3. Felhasználói dokumentáció

3.1 A webalkalmazás ismertetése röviden

A felhasználók számára ez egy kétoldalú program. Hasznos edzőterem tulajdonosok és fitness rajongók számára is. Könnyedén lehet kezelni edzőtermeit és személyi edzőit, míg a másik oldalon egyszerű módon vásárolható edzőtermi bérlet és az adott edzőteremben az összes elérhető személyi edzőt is láthatja.

3.2 Szükséges hardver eszközök

A program elérhető bármilyen eszközön amin van internet elérés. Legyen az PC, laptop, tablet vagy okostelefon.

3.3 Szükséges szoftver eszközök

A program használatához szükség van bármilyen modern böngészőre, de a tökéletes működéshez a Google Chrome-ot, Operát, vagy Safarit ajánljuk.

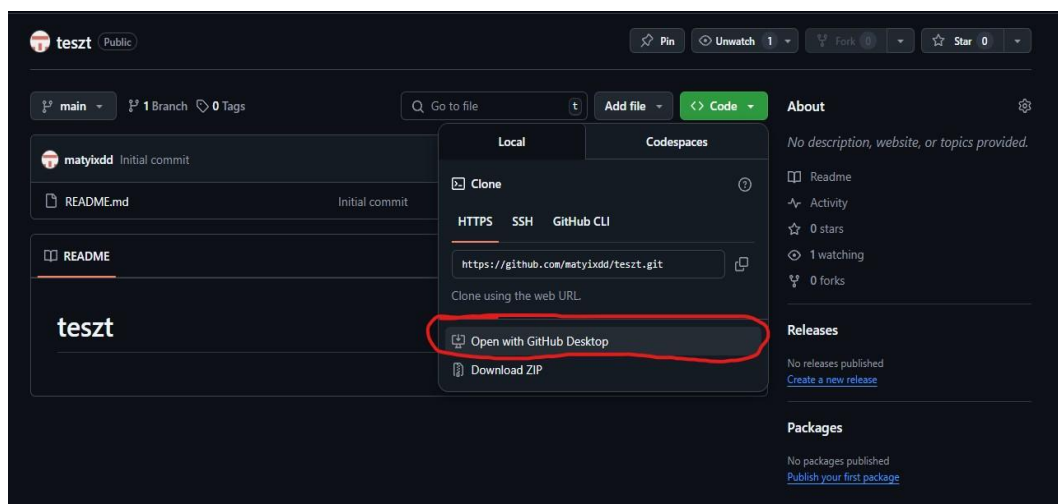
3.4 Az alkalmazás beüzemelése, működtetése

Egyenlőre nem használunk online webszerver hosztolást, szóval a weboldalunk csakis XAMPP lokális szerveren tekinthető meg. A következő lépések szigorú betartásával indítható el a program:

GitHub weboldalon megnyitja a vizsgaremek repository-nak a "Code" fülét.

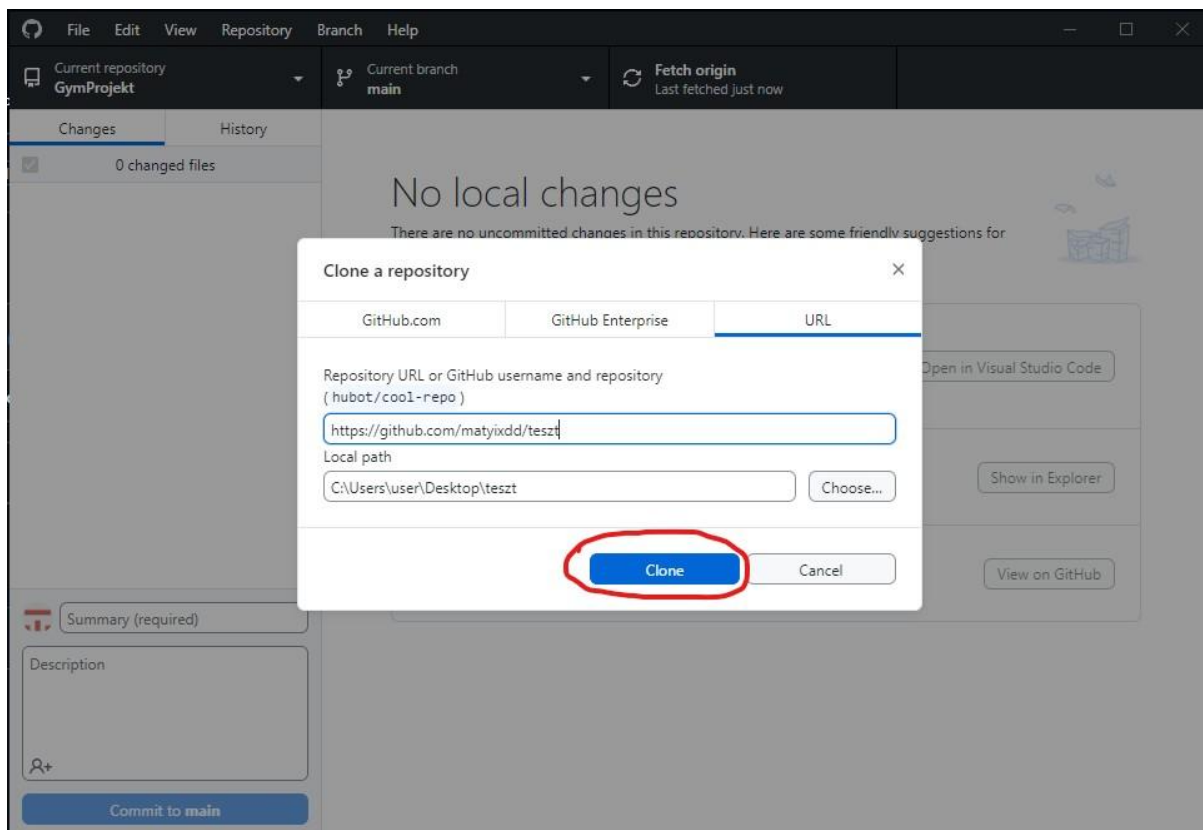
Ezután rákattint a zöld "Code" gombra, ennek local fülén belül rákattint az "Open with GitHub desktop" gombra.

Ez a legfontosabb lépés. Ha csak ZIP-ként tölti le, nem fog működni.



Ha még nincs letöltve a GitHub Desktop applikáció számítógépére, letölti.

Ezután tetszőleges helyre leklónozza a repositoryt.



Következő lépés: Megnyitja a repository mappáját, benne a "GymProjekt mappát". Ebben a mappában van egy "xampp" mappa, ezt megnyitja.

GymProjekt				Keresés: GymProjekt
Név	Módosítás dátuma	Típus	Méret	
Adatbázis	2025. 04. 25. 22:03	Fájlmappa		
tests	2025. 04. 26. 21:42	Fájlmappa		
vendor	2025. 04. 25. 22:36	Fájlmappa		
xampp	2025. 04. 10. 21:01	Fájlmappa		
composer.json	2025. 04. 26. 21:31	JSON forrásfájl	1 KB	
composer.lock	2025. 04. 26. 21:31	LOCK fájl	63 KB	
composer.phar	2025. 04. 25. 22:18	PHAR fájl	3 042 KB	

Az xampp mappában megkeresi a "setup_xampp.bat" nevű fájlt és ezt lefuttatja.

Ezt fogja látni:


```
##### START XAMPP TEST SECTION #####
[XAMPP]: Test php.exe with php\php.exe -n -d output_buffering=0 --version ...
PHP 8.0.30 (cli) (built: Sep 1 2023 14:15:38) ( ZTS Visual C++ 2019 x64 )
Copyright (c) The PHP Group
Zend Engine v4.0.30, Copyright (c) Zend Technologies
[XAMPP]: Test for the php.exe successfully passed. Good!
##### END XAMPP TEST SECTION #####

#####
# ApacheFriends XAMPP setup win32 Version                                     #
#-----#
# Copyright (c) 2002-2025 Apachefriends 8.0.30                               #
#                                     #
#-----#
# Authors: Kay Vogelgesang <kvo@apachefriends.org>                           #
#          Carsten Wiedmann <webmaster@wiedmann-online.de>                  #
#####

Do you want to refresh the XAMPP installation?
Soll die XAMPP Installation jetzt aktualisiert werden?

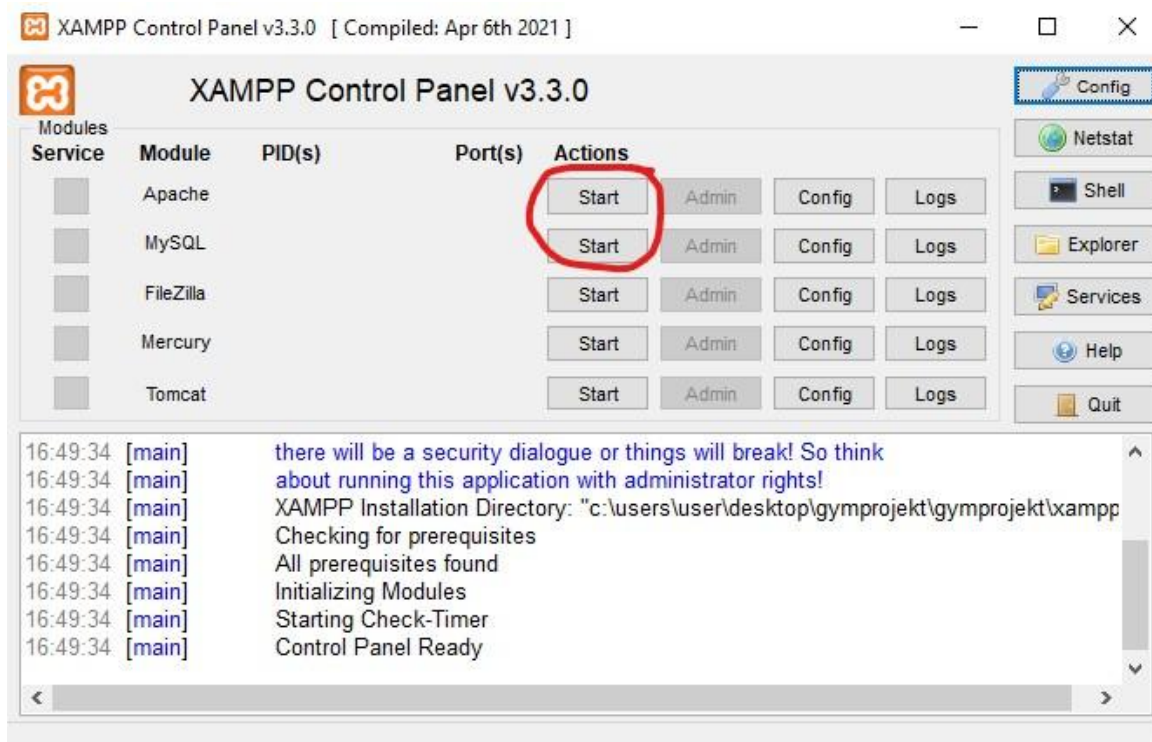
1) Refresh now! (Jetzt aktualisieren!)
x) Exit (Beenden)
```

Az utasítás szerint begépeli hogy 1 és entert nyom. Megvárja hogy végig menjen a parancssor majd ha végzett kiléphet ebből. Ezzel lefrissítette az XAMPP-ot hogy tökéletesen működjön a gépén.

Ezután az xampp mappában megkeresi és lefuttatja az "xampp-control.exe" alkalmazást.

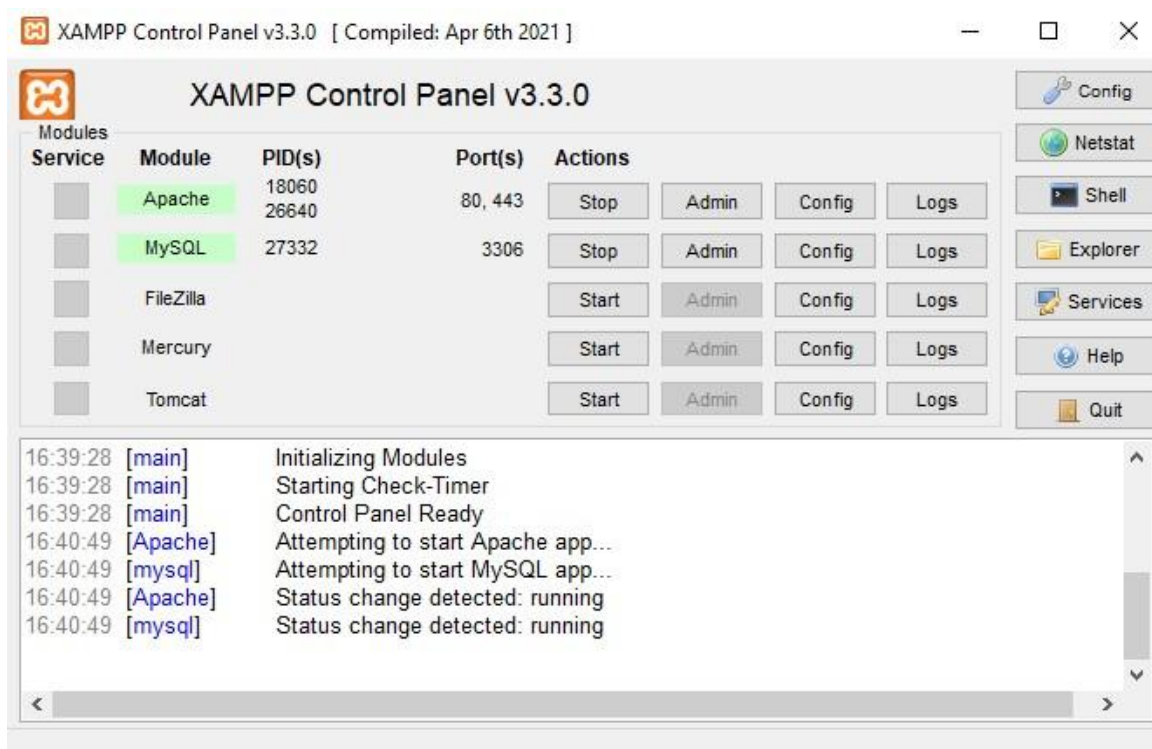
	setup_xampp.bat	2013. 03. 30. 13:29	Windows kötegfájl	2 KB
	test_php.bat	2020. 11. 29. 14:38	Windows kötegfájl	2 KB
	uninstall.dat	2025. 02. 27. 14:17	DAT fájl	258 KB
	uninstall.exe	2025. 02. 27. 14:17	Alkalmazás	6 432 KB
	xampp_shell.bat	2025. 02. 27. 14:10	Windows kötegfájl	2 KB
	xampp_start.exe	2013. 03. 30. 13:29	Alkalmazás	116 KB
	xampp_stop.exe	2013. 03. 30. 13:29	Alkalmazás	116 KB
	xampp-control.exe	2021. 04. 06. 13:38	Alkalmazás	3 290 KB
	xampp-control.ini	2025. 02. 27. 14:17	Konfigurációs beá...	1 KB
	xampp-control.log	2025. 04. 10. 21:01	Szöveges dokume...	9 KB

Ezt fogja látni:

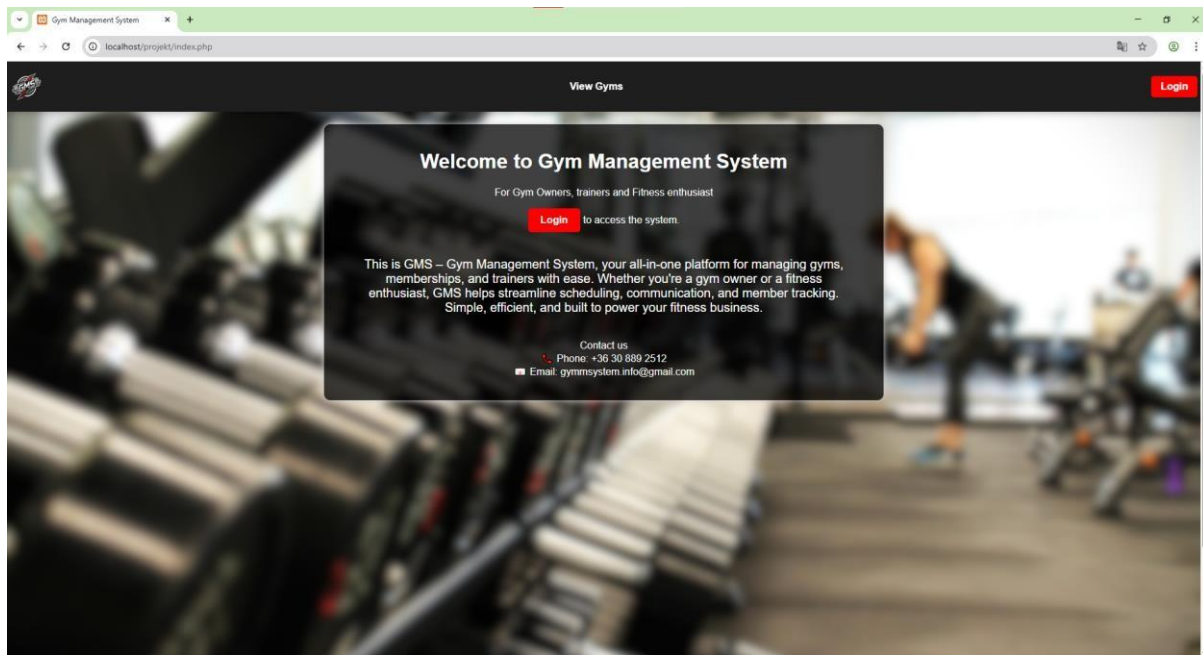


Itt a "Start" gombbal elindítja az Apache és MySQL szervereket.

Ha minden lépést pontosan követett, elindulnak a szerverek:

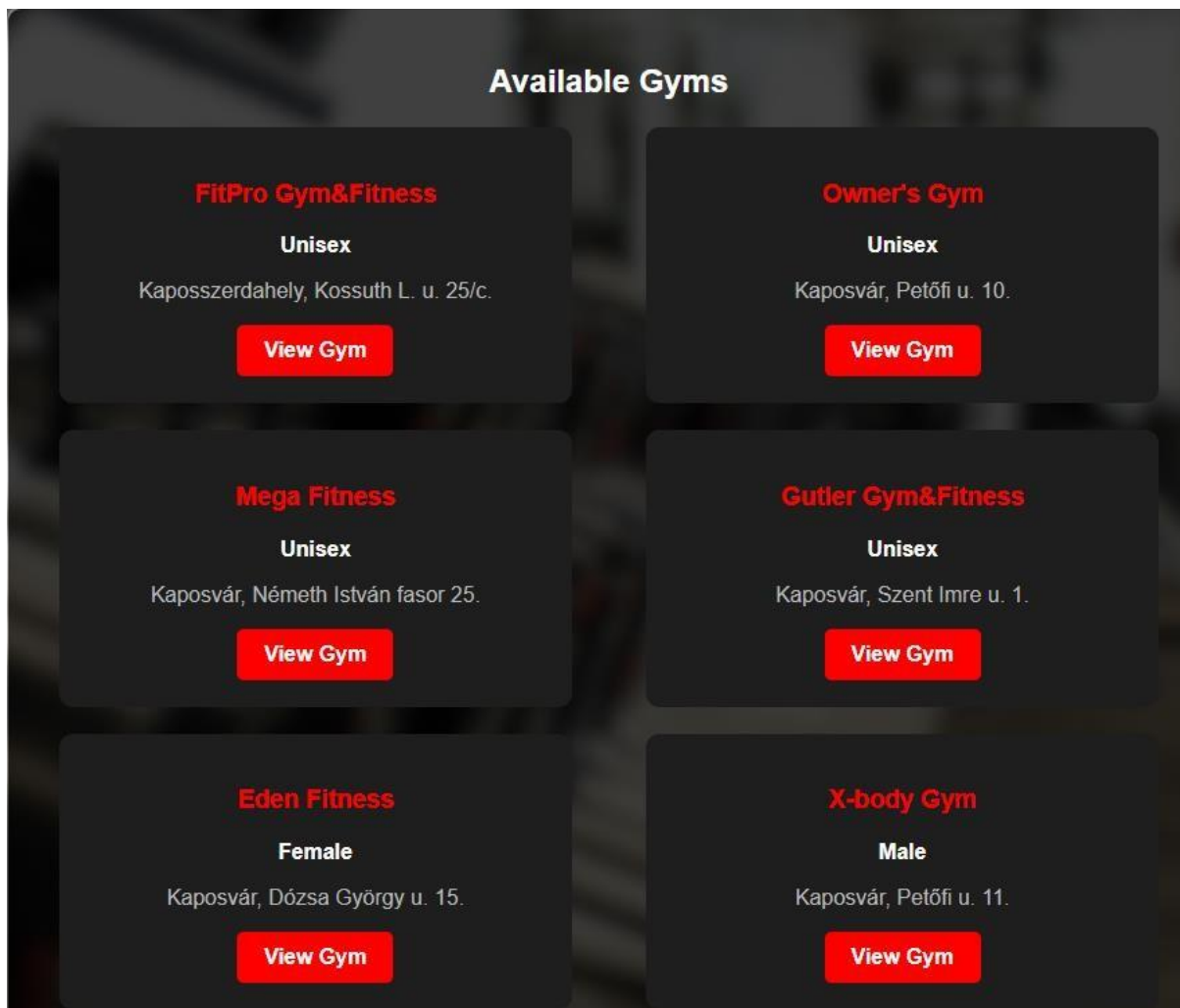


Ezután a weboldal megtekinthető, ha webböngészőjébe beírja, hogy `http://localhost/projekt/index.php`



3.5 Program használata

A weboldalon bejelentkezés nélkül is megtekinthető az összes elérhető edzőterem és azon belül a személyi edzők az oldal tetején lévő "View Gyms" gombra kattintva, de bérletet már nem lehet így vásárolni.



Jobb felső sarokban a "Login" gombra kattintva jelentkezhet be, vagy regisztrálhat a felhasználó. A weboldal 2 féle felhasználót különböztet meg: "sima" felhasználót és edzőterem tulajdonost, ezért fontos, hogy a tulajoknak külön oldalon kell regisztrálniuk.

Login to Gym Management System

Login

Don't have an account yet? [Register here](#)

Register



Register

Already have an account? Login [here](#)

Want to manage a gym? [Register as a Gym Owner](#)

Register as Gym Owner



Register as Owner

Already have an account? [Login here](#)

Just want to join a gym? [Register as a Member](#)

Sima felhasználónak nem változik sokat az oldal bejelentkezés után. 2 új funkciót tud használni: edzőtermi bérlet vásárlás és saját profil megtekintése, ahol láthatja hogy milyen edzőtermekbe van bérlete.

Purchase Membership for Owner's Gym

Your Name

7 Days

Enter Card Details

Card Number

Expiry Date (MM/YY)

CVV

Purchase Membership

Profile

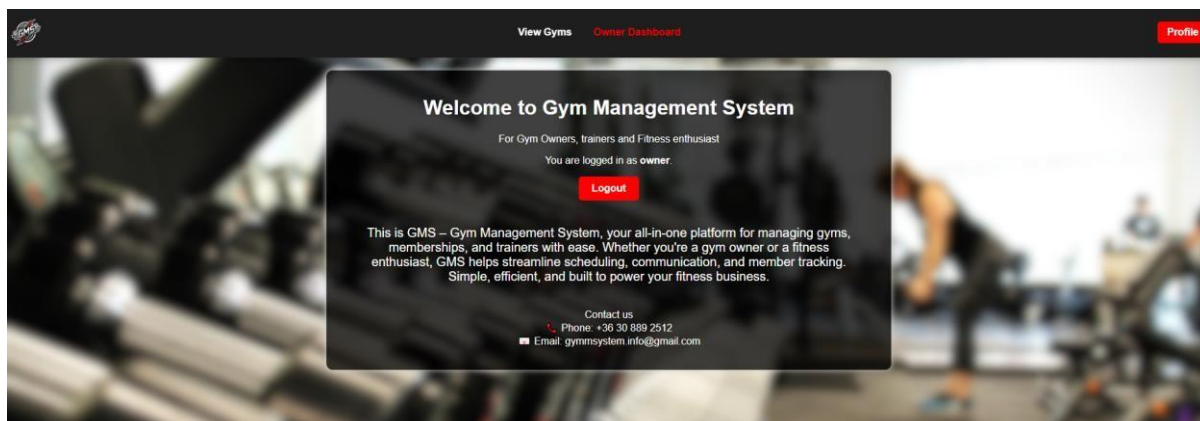
Logged in as user

My memberships

FitPro Gym&Fitness Unisex Kaposzserdahely, Kossuth L. u. 25/c. View Gym	Owner's Gym Unisex Kaposvár, Petőfi u. 10. View Gym
---	---

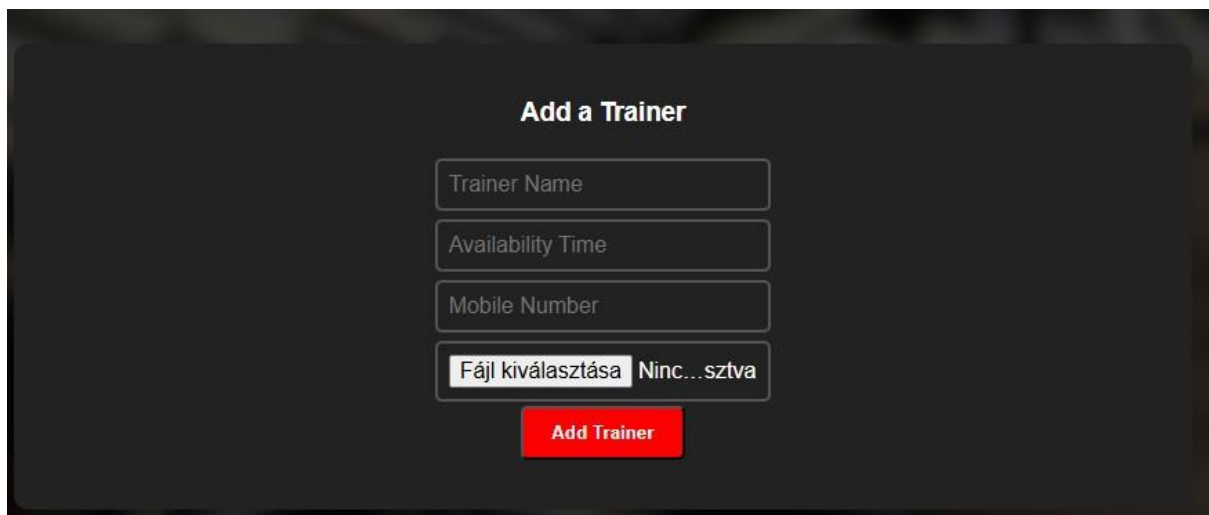
Logout

Ha edzőterem tulajként jelentkezik be, már az első oldalon látszódik a változás: megjelent egy "Owner Dashboard" gomb a navigációs sávon.



Az oldalon lehet új edzőtermet felvenni:

Új edzőket hozzáadni:



Add a Trainer

Trainer Name

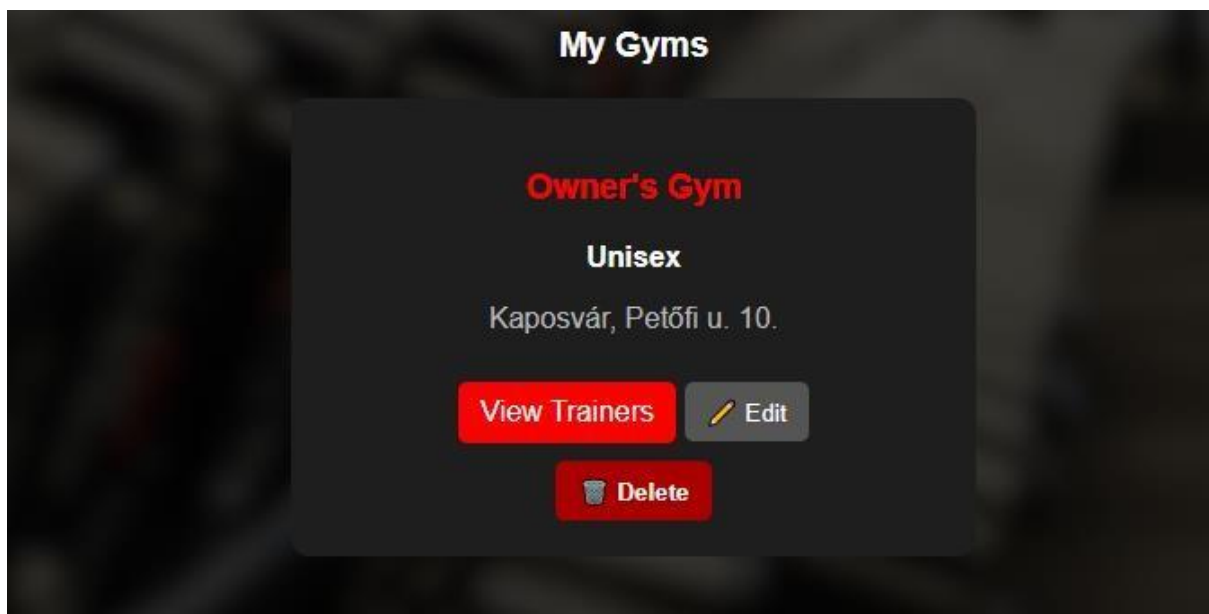
Availability Time

Mobile Number

Fájl kiválasztása Ninc...sztnva

Add Trainer

Kezelni meglévő edzőtermeket:



My Gyms

Owner's Gym

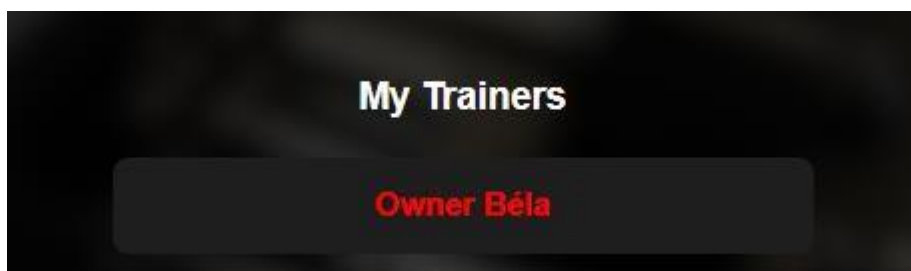
Unisex

Kaposvár, Petőfi u. 10.

View Trainers  **Edit**

 **Delete**

Megnézni a hozzáadott személyi edzők listáját:



My Trainers

Owner Béla

Nevükre kattintva szerkeszteni adatait, változtatni a hozzárendelt edzőtermeit vagy teljesen törölni a rendszerből:

Owner Béla

📞 Mobile: +5555555555



Assign to Gyms:

☒ Owner's Gym

 Save Changes

Edit Mobile Number

 Update

 Delete Trainer

3.6 Lehetséges hibajelzések

3.6.1 Invalid credentials

Bejelentkezésnél helytelen felhasználónév vagy jelszó esetén "Invalid credentials" üzenetet kap vissza.

Login to Gym Management System

admin

.....

Login

Invalid credentials

Don't have an account yet? [Register here](#)

3.6.2 Passwords do not match

Regisztrálásnál, ha a jelszó és a jelszó megerősítése mezőbe nem ugyanazt írja, "Passwords do not match!" hibajelzést kap.

Register

teszt

.....

.....

+4342332

2005 . 01 . 01 .

Register

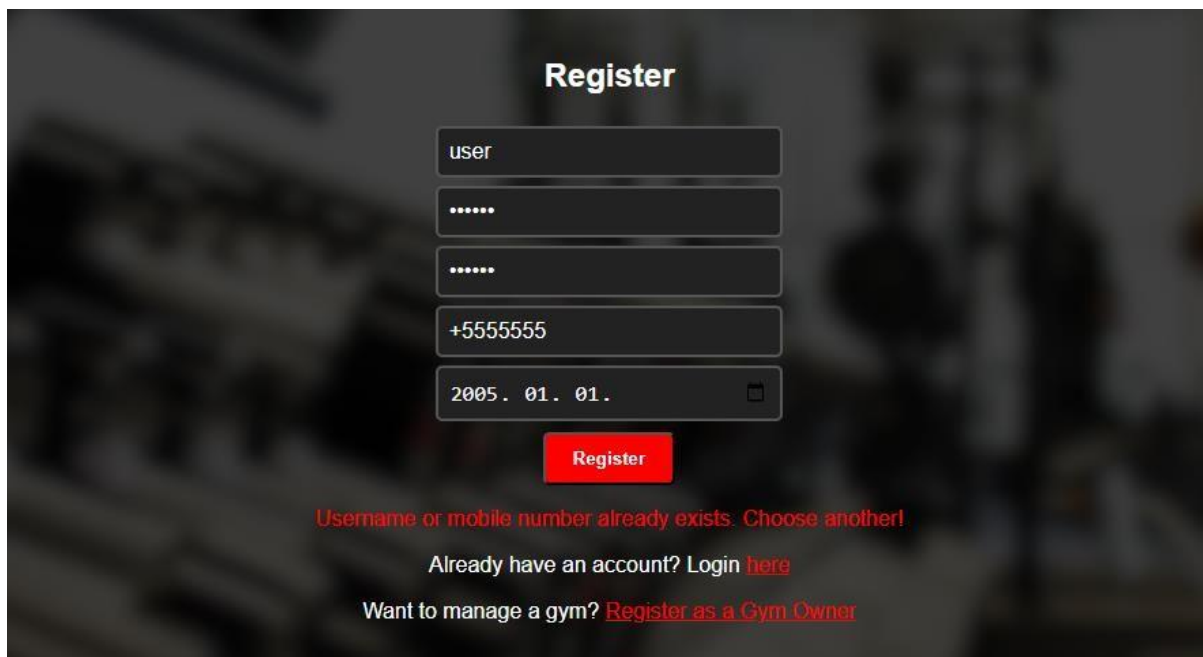
Passwords do not match!

Already have an account? Login [here](#)

Want to manage a gym? [Register as a Gym Owner](#)

3.6.3 Username or mobile number already exists

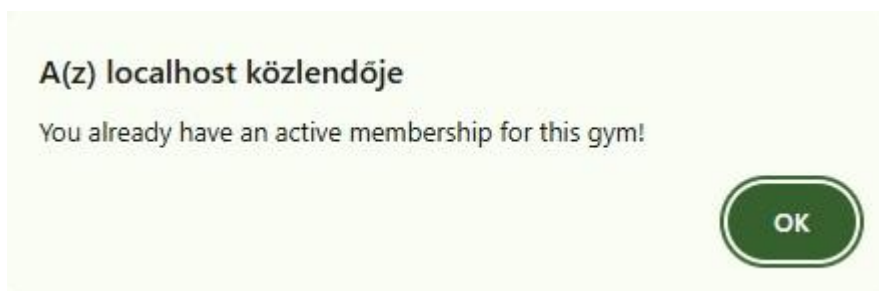
Regisztálásnál, ha már létező felhasználónevet vagy telefonszámot ad meg, "Username or mobile number already exists. Choose another!" üzenetet kap vissza.



The image shows a 'Register' form on a dark background. The form has five input fields: 'user' (containing 'user'), a password field (containing '*****'), a confirm password field (containing '*****'), a phone number field (containing '+5555555'), and a date field (containing '2005 . 01 . 01 .'). Below the fields is a red 'Register' button. Under the button, there is a red error message: 'Username or mobile number already exists. Choose another!'. Below the error message, there are two links: 'Already have an account? Login [here](#)' and 'Want to manage a gym? [Register as a Gym Owner](#)'.

3.6.4 You already have a membership for this gym

Ha megpróbál edzőtermi bérletet venni egy olyan edzőterembe, ahol már van aktív bérlete, "You already have an active membership for this gym!" üzenetet kap vissza.



3.7 Elérhetőség információ kéréshez

A webalkalmazás érdekében készítettük az alábbi e-mail címet ahol segítséget tudunk nyújtani kérdések felmerülése esetén:

gymmsystem.info@gmail.com

4. Összefoglalás

4.1 Munka felosztása

A hatékony együttműködést Discordos hívások és a Github verziókövető rendszer biztosította a projekt során.

Projektvezető, programozó (Göndör Mátyás): Adatbázis elkészítése, Owner dashboard, login és registration kód megírása, csapat munkájának megfelelő elosztása.

Tesztelő, programozó (Fekete Ákos András): Edzőterem lista, profil, unit tesztelés és egyéb kisebb dolgokban segítség nyújtás.

Designer, Dokumentáció megírása (Csordás Bence): A webalkalmazás kinézetének tervezése, a dokumentáció megvalósítása.

4.2 A webalkalmazás jövője

Tovább fejlesztés után egy edzőterem tulajdonos ismerősünknek adnánk át az alkalmazást. Reménykedünk benne, hogy a jövőben egy webszerver segítségével véglegesen kiadhatjuk az internetre.